



Convergent Portal Build Protocol – Concept2Cure Platform

Prepared for Developer Agent Claude Opus 4.5 (GitHub Codespaces environment)

Overview

This protocol details the **Concept2Cure** next-generation client portal build, converging multiple modules into a unified workspace. It integrates the **“UI Enhancement Shell + Structure” project steps (1–10)** and aligns with the **Convergent Canvas** architecture and **Lumen Cortex** design principles. The goal is to guide a developer agent (Claude 4.5) through building and stabilizing an enterprise-grade portal UI from scratch in a fresh Codespace, using only the repository and AI assistance. Each step below is a **Codespace-compatible build action**, ensuring high development velocity while meeting regulatory-grade quality.

Key objectives and context:

- **Multi-tenant Modular Portal:** A single portal interface serves both CRO super-admins and biotech client users, with role-based views and dynamic module loading.
- **AI-Integrated Workspace:** The portal includes an AI assistant and intent-driven navigation for intelligent user interactions.
- **Regulated Domain Focus:** Accommodate various regulatory contexts (FDA, EMA, PMDA) and product types (biotech pharma vs. medical device), adapting UI and modules accordingly.
- **Modern UI & Design Cohesion:** Employ a polymorphic layout, **cache-first AI integration**, and a **warm luxe brand palette** to create a sleek, responsive, and branded user experience.

Architecture Alignment: Convergent Canvas & Lumen Cortex

Before diving into implementation steps, the portal design must reflect **Convergent Canvas** architecture and **Lumen Cortex** principles:

- **Polymorphic Layouts:** The UI layout is flexible and context-aware. The portal shell and component panels morph based on user role and context (e.g. CRO admin vs biotech client) without requiring separate apps. A CRO user might see a multi-organization dashboard, whereas a biotech user sees only their own organization context – all within the same component structure.
- **Cache-First AI Integration:** The platform tightly weaves in AI capabilities (Claude-powered assistant) with an emphasis on performance. Shared context and results are cached on the client to minimize redundant calls. For example, the portal maintains a local context state for the current org/program/study selection and recent AI query results, so the assistant can retrieve info quickly without always calling the server ¹. This design ensures the AI features feel instant and context-rich.

- **Intent Engine Routing:** Navigation can be driven by user intent (not just clicks). An **Intent Engine** listens to natural language commands or contextual triggers to route the user to the right module or page. For instance, if the user asks the AI “Show the IND submission status,” the system should interpret this and navigate to the IND Wizard module or surface relevant data. The routing logic will map intents to module routes using the module registry (defined later), enabling a seamless handoff from AI suggestions to UI actions.
- **Warm Luxe Brand Palette:** The UI adheres to a consistent, upscale visual style. This means using warm, muted tones (e.g. deep blues or gold accents on ivory backgrounds) and high-quality typography to convey a premium feel. In practice, we will apply a custom theme to components (via Tailwind CSS or Fluent UI theme settings) so that classes like `text-primary` and `bg-primary` reflect the Concept2Cure brand (a “warm luxe” hue) across the app. Every module and common element (buttons, headers, icons) should use this palette for a unified look and feel.

With these principles in mind, we proceed through the 10 build steps, each contributing to the final convergent portal. Each step includes file-level directives, component paths, and interface specs where applicable, ensuring the plan is **executable in a Codespace** with the help of GitHub Copilot or Claude.

Step 1: Define Portal Contract and Module Policy

Goal: Establish a clear contract for what the portal includes and under which conditions, by defining TypeScript interfaces and configuration objects. This “portal policy” will map roles and context (agency, study type, product) to available modules and features.

- **Create** `client/src/portal/policy/portalPolicy.ts`: This will house TypeScript interfaces and constants that formalize the portal's structure. Key interfaces:
- `ModuleConfig` – describing each module's identity and availability. For example:

```
export interface ModuleConfig {
  id: string;           // e.g. "ind-wizard"
  name: string;         // "IND Application Wizard"
  route: string;        // URL route, e.g. "/ind"
  icon?: React.ReactNode; // Icon for navigation (if any)
  roles: string[];      // Roles allowed (e.g. ["Admin","Editor"])
  agencies: string[];   // Supported agencies (e.g.
  ["FDA","EMA","PMDA"])
  studyTypes: string[]; // Applicable study types (e.g.
  ["IND","CER","510k"])
  productTypes: string[]; // Applicable product types (e.g.
  ["Drug","Device"])
}
```

- `PortalPolicy` – defining the overall configuration, possibly as a collection (array or map) of `ModuleConfig` entries (or grouping by role/agency). For instance, `PortalPolicy.modules`

could be an array of all module configs, or you might separate them by category (dashboard modules, workflow modules, admin modules). This acts as the **registry of modules and features** the portal can present.

- **List available modules in the policy:** Populate `PortalPolicy` with all initial modules to integrate. For example:
- **IND Wizard:** For Investigational New Drug applications (FDA context) – `id: "ind-wizard"`, route `/ind`, roles: all editors/admins, agencies: FDA (perhaps EMA if analogous process exists), studyTypes: IND, productTypes: Drug/Biologic.
- **Trial Vault:** A trial document repository – `id: "trial-vault"`, route `/vault`, roles: all (document access for all collaborators), agencies: all (document management is universal), studyTypes: all, productTypes: all.
- **CSR Intelligence:** Clinical Study Report analyzer – `id: "csr-intelligence"`, route `/csr`, roles: all, agencies: all (CSR applicable globally), studyTypes: any trial requiring CSR, productTypes: Drug/Biologic (primarily).
- **Study Architect:** Protocol design module – `id: "study-architect"`, route `/protocol`, roles: all editors, agencies: all, studyTypes: all (every study needs a protocol), productTypes: all.
- **Analytics Dashboard:** Portfolio analytics – `id: "analytics"`, route `/analytics`, roles: all, agencies: all, studyTypes: all, productTypes: all (a generic metrics module).

These are examples; include any other modules (like a **Tasks manager**, etc.) as needed. By enumerating them in one place, the portal can reference this registry for rendering navigation and enabling features.

- **Role and context mapping:** In `portalPolicy.ts`, also define structures for roles and perhaps enums for agencies or product types. For example, an enum or union for roles (`"Admin" | "Editor" | "Viewer"`) and similarly for regulatory agencies (`"FDA" | "EMA" | "PMDA"` etc.). This helps ensure consistency in checking permissions across the app. You might encode a mapping like `const ModulesByRole: Record<Role, ModuleConfig[]>` to list which modules each role can access, or conversely each `ModuleConfig` might already contain its roles (as above). The **Portal contract** essentially means that for any given user context (role + agency + product), the allowed modules are predetermined by this policy.
- **Polymorphism via policy:** This setup allows the UI to **dynamically adapt**. For example, if a user is working on a **CER (Clinical Evaluation Report)** for a **medical device** in the EU (EMA), the policy might exclude the IND Wizard and include a “CER Module” if defined. The developer agent should encode these differences in the policy (even if some modules are placeholders for now). This approach aligns with Convergent Canvas by treating modules as plug-ins that the portal pulls in or leaves out based on context.

By clearly defining the portal’s module contract in code, we create a single source of truth that will guide the navigation, context logic, and even the AI assistant’s knowledge of what it can do (the assistant can reference this registry to know what modules exist and what to suggest).

Step 2: Implement Portal Layout Shell & Navigation Structure

Goal: Create the foundational layout component (`PortalShell`) that frames the portal UI, including global navigation (sidebar, header) and content area. This shell will enforce a consistent structure and allow nested module pages to render within it.

- **Create a PortalShell component** (e.g., `client/src/layouts/PortalShell.tsx`): This component will define the main page structure. It should include:
 - **Side Navigation:** A vertical menu or sidebar that lists high-level navigation and context switches. For a CRO admin user, this side nav includes an organization switcher (listing multiple client orgs) and possibly other global nav items (like “Dashboard”, “Projects”, “Admin”, etc.). For a biotech client user, the side nav might be simpler (only their projects/modules, since they have a single org). Use the module registry from Step 1 to populate module links dynamically – e.g., iterate over `PortalPolicy.modules` and render a `Link` for each module the current user should see.
 - Implement context-based sections in the sidebar. For example, a CRO might first select an **Organization**, then see that org’s **Programs**, then a Program’s **Studies**. In Sprint-6, a rudimentary version of this was done: the side nav fetched organizations and allowed drilling down ² ³. We will build on that. Use the `PortalContext` (to be created in Step 3) to keep track of selected org/program/study and update it when the user clicks a nav item ⁴. Each level of selection refines what’s shown next (programs list after org selection, etc.), and ultimately the study/workspace is selected.
 - For module navigation (like going to Vault, IND Wizard, etc.), you might include those as separate sections or menu items in the sidebar or top nav. E.g., under a selected study, show links to relevant modules (“Trial Vault”, “Analytics”, etc.). Ensure the **active module** or page is highlighted in the nav.
 - **Top Header (Context Bar):** A top bar that displays the current context and provides utility buttons. This could show the current organization’s name (and maybe type/tag) and user account menu. For example, the client portal code displays the client org name and type with an icon ⁵. Also include a global search input or icon (for searching projects or documents) and a button to open the AI assistant panel. The header is also a good place to put a “Return to [parent org]” link for when a CRO admin is impersonating a client’s view ⁶. Using a consistent header ensures even when content switches, the user always knows where they are (context breadcrumb) and has access to global tools (search, help, AI, notifications).
 - **Main Content Outlet:** Reserve the majority of screen space for module content or pages. In React Router terms, `PortalShell` will render an `<Outlet />` for nested routes ⁷. We will nest all portal routes (dashboard, projects, study workspace, etc.) inside `PortalShell` so that the sidebar and header remain persistent. For example, in the app’s route config, wrap portal routes like:

```
<Route element={<PortalShell />}>
  <Route path="/dashboard" element={<ClientPortalLanding/>}/>
  <Route path="/projects" element={<ProjectsPage/>}/>
</Route>
```

```

<Route path="/studies/:studyId/*" element={<StudyWorkspace/>}/>
<Route path="/admin/*" element={<AdminConsole/>}/>
</Route>

```

This way, navigating to any portal sub-page will automatically display within the PortalShell layout

8 .

- **Responsive design considerations:** The PortalShell layout should be responsive. The sidebar can be collapsible or horizontal on smaller screens. Use CSS grid or flexbox to allow the content area to resize as needed. For example, in a desktop view, you might have a fixed-width sidebar and a fluid content area; on tablet, the sidebar might become an icon strip or slide-out menu.
- **Apply warm luxe styling:** Style the PortalShell and nav using the brand palette. For instance, the sidebar background could be a warm neutral tone, with active link highlights in the brand's accent color. Leverage Tailwind configuration or Fluent UI theming. Ensuring these styles in the shell means all pages consistently reflect branding. *Example:* define Tailwind classes like `.bg-primary` and `.text-primary` to match brand colors, and use them for buttons and highlights. The header and sidebar can have subtle shadows and gradients to convey a polished look (luxe feel).
- **Integrate global elements:** The PortalShell is also a good place to mount components that should persist across modules, such as the AI assistant trigger (floating button or a sidebar section) and a notification toasting system for success/error messages. This ensures continuity—e.g., the AI assistant panel can remain open as the user navigates between modules, thanks to being rendered at the shell level.

By implementing the PortalShell and nav, we establish a consistent **layout structure** that all modules plug into. This addresses the “layout” and basic “portal contract” part of the UI Enhancement project, providing the skeleton on which specific features will be built.

Step 3: Integrate Portal Context and State Management

Goal: Set up a React Context for portal-specific state (organization, program, study, etc.) and connect it with an integration layer to share context across components and with the AI. This “context layer” ensures the UI and AI always know the current working context and can react to changes.

- **Create PortalContext:** In `client/src/context/PortalContext.tsx`, define a React context to hold the portal state. This should include at least:

```

export interface PortalCtx {
  orgId?: string;
  programId?: string;
  studyId?: string;
  userRole?: string;
  region?: string;
  // ... any other context, e.g., currentProjectId, activeModule, etc.
  set: (update: Partial<PortalCtx>) => void;
}

```

```

}
export const PortalContext = createContext<PortalCtx>({ set: () => {} });

```

Also create a `PortalProvider` component that uses `useState` internally and provides a `set` function to merge updates ⁹. This allows any part of the app to call `portalCtx.set({ orgId: "xyz" })` to update the context. Initialize the context state from the logged-in user's default org or role if available (e.g., on load, set `userRole` from Auth info, and if a biotech user only has one org, set that orgId by default).

- **Context usage in navigation:** Use `PortalContext` in the `PortalShell`'s sidebar and other components to read/update selections. For example, in the Org/Program selector sidebar, when a user clicks an organization link, call `portalCtx.set({ orgId: o.id, programId: undefined, studyId: undefined })` to set the current org and reset deeper context ⁴. This triggers the effect hooks to load programs for that org. Similarly, selecting a program sets `programId` and resets `studyId`, etc. The context provider will make these values available app-wide.
- **Shared context for modules (Integration Layer):** In addition to `PortalContext`, implement or leverage a **Module Integration Layer** context that was hinted in the code (`useModuleIntegration`). This is a higher-level context/service that allows modules and the AI assistant to share data beyond just IDs. For instance, after setting `orgId` in `PortalContext`, you might fetch the full org object and share it:

```

// Pseudocode:
const { shareContext } = useModuleIntegration();
useEffect(() => {
  if (orgId) {
    // fetch organization details by orgId (or derive from a cached list)
    shareContext('organization', orgId, { organization: orgDetails });
  }
}, [orgId]);

```

In the Sprint-6 example, once an org was selected, they shared the org data with other modules ¹⁰. The integration layer can maintain a `sharedContext` object mapping context types to data (e.g., `'organization': { [id]: {organization: {...}, parentOrganization: {...} } }`). The AI assistant panel then can retrieve this context. For example, in `AIAssistantPanel`, they call `getRelevantContext()` to pull the latest shared context for inclusion in prompts ¹ ¹¹. We need to ensure our context updates feed into this `sharedContext`.

- **Include role and region in context:** To support adaptive UI by role and agency, ensure these are part of `PortalContext` (and `sharedContext`). For example, set `portalCtx.userRole` from the `AuthContext` (which might have the user's role for the org) ¹², and determine `portalCtx.region` or `agency` based on the org or study (perhaps each study or org has a region attribute – if not, allow user to pick a regulatory region which could influence available modules). This way, when rendering modules or pages, you can filter by these context values against the `ModuleConfig` (from Step 1) to decide what to show.

- **Leverage AuthContext:** The app likely already has an `AuthContext` providing authentication info (token, user data) ¹³. Integrate that with `PortalContext` initialization (for example, use the `AuthContext` to get the current user's org or role on load). Also use `AuthContext` for API calls as seen in existing code (passing `Authorization: Bearer ...` headers) ². Ensuring `PortalContext` and `AuthContext` work together will let you seamlessly call backend endpoints for data when context changes (e.g., load programs when `orgId` changes ¹⁴).
- **State caching and performance:** `PortalContext` acts as a client-side cache for the current selections and possibly objects. For instance, if a user selects an org, the org details can be stored so that multiple components don't all refetch it. The **cache-first integration** principle means we trust this context (and any associated caches like in the integration layer) as the first source of truth, only hitting the backend when needed or on context switches. The developer agent should implement appropriate `useEffect` hooks to fetch data on context changes (as done in the `SideNav` code) and then **store it in context or a cache** for reuse.

By implementing the `PortalContext` and integration layer, we enable **reactive UI**: changes in one part (e.g., selecting a new study in a dropdown) instantly propagate to all components (the dashboard cards show that study's info, etc.), and provide the AI assistant with up-to-date context. This fulfills the "context layer" step of the project, establishing a robust state management strategy.

Step 4: Establish Module Registry and Intent Routing

Goal: Implement the **module registry** defined in the portal policy into functional code and set up an intent-driven routing mechanism. This will allow dynamic module loading and intelligent navigation based on user input or AI commands.

- **Instantiate the module registry:** Using the definitions in `portalPolicy.ts` (from Step 1), create a constant list of modules. For example, `export const MODULES: ModuleConfig[] = [ {...IND Wizard config...}, {...Trial Vault config...}, ... ];`. Ensure each entry has the necessary metadata (id, route, roles, etc.). If needed, also export categorized lists (e.g., `DASHBOARD_CARDS` for modules that have dashboard quick-cards, versus full-page modules). This registry will be imported by various parts of the app:
 - In the `PortalShell/Sidebar` to generate navigation links for modules.
 - In the `Dashboard` to conditionally render quick access cards (e.g., only show `AnalyticsQuickView` if the "analytics" module is enabled for this context).
 - In the AI assistant logic to know what modules exist and maybe suggest them.

For instance, the `ClientPortalLanding` component already imports `MODULES` from the integration layer ¹⁵ (it suggests an array was available). Make sure that `MODULES` reflects our `PortalPolicy` definitions.

- **Dynamic navigation using ModuleConfig:** Refactor or implement the UI navigation to be data-driven. Rather than hardcoding routes in JSX, loop through `MODULES` filtered by context:

```
{MODULES.filter(m =>
  m.roles.includes(portalCtx.userRole!) &&
  m.agencies.includes(portalCtx.region!) &&
  m.productTypes.includes(currentProductType)
).map(module => (
  <Link key={module.id} to={module.route}>{module.name}</Link>
))}
```

This ensures the user only sees links they can actually use. For example, a **Viewer** role might not see the Admin console link, because the Admin module's `roles` might exclude "Viewer". Or if `region` is "EMA" and there's a module only for FDA context, it gets filtered out. This mechanism is a direct application of the PortalPolicy rules at runtime.

- **Implement the Intent Engine:** Create a utility (maybe in `client/src/portal/IntentRouter.ts` or within the AI assistant component) to handle intent-based commands. This can be simple initially:

- Map certain keywords or phrases to module routes or actions. For example:

```
const intentMap = {
  "open ind": "/ind",
  "IND Wizard": "/ind",
  "open vault": "/vault",
  "document vault": "/vault",
  "analytics": "/analytics",
  "open tasks": "/tasks",
  // ... etc
};
```

The keys can be lowercase phrases for matching user input.

- Expose a function like `handleIntentCommand(command: string): boolean` that checks if the command matches any key in the map. If so, use React Router's navigation (or `window.location`) to redirect to the mapped route, and return true (indicating the intent was handled).
- Integrate with AI Assistant: In the `AIAssistantPanel`, after receiving a response from the model, you could parse it for known triggers. Alternatively, you might allow the AI to output a special format (like a JSON or a tag in its message indicating a navigation action). But a simpler approach: if the user's prompt starts with a known verb like "open" or "go to", you can attempt to handle it as an intent. Also, the assistant could be trained to sometimes respond with suggestions like "(Click **Analytics** module)" which the user can then click. For now, focus on the plumbing: ensure that if an intent is recognized, the portal will navigate accordingly.

- **Example usage:** If user types "open the IND Wizard module" into the assistant, after the assistant responds (or even before calling the model), intercept that query. The phrase "open ... IND Wizard" triggers the intent engine, which then navigates to `/ind`. The assistant might still answer, but the user is also taken to the IND module page. This creates a fluid experience where conversational commands drive the UI.
- **Pre-loading modules (optional):** As part of a "cache-first" approach, consider preloading certain module components when the portal loads or context switches. For instance, if the user has access to the IND Wizard, you might preload its code split bundle in the background, so that when they navigate via intent or click, it opens instantly. React's `Suspense` and dynamic import can be used here. This step is an optimization that can be done after basic functionality is in place.
- **Verification of routing:** After setting up the registry and intent logic, test that all module links work (clicking nav or quick links opens the correct component inside PortalShell) and that the intent mapping covers key phrases. Because this is meant for an AI agent to execute, it's important the instructions for mapping intents are clear and deterministic.

By the end of this step, the **Module Registry** is fully realized in code and the portal supports **intent-based navigation**. This directly supports the Lumen Cortex idea of an intelligent interface – the system not only has a list of capabilities (modules) but can route to them in response to user intent, whether expressed via UI clicks or natural language commands.

Step 5: Build Dashboard Presets and Overview Page

Goal: Develop the main **Portal Dashboard (Landing) page** that provides a high-level overview and quick access to key modules. This step integrates multiple UI components: project summary, quick-access panels, next actions, etc., into a cohesive homepage for the portal.

- **Assemble the Client Portal Landing page:** Create or refine `ClientPortalLanding.jsx` (the landing dashboard). This page will be shown at the route `/dashboard` or `/client-portal` after login, inside the PortalShell. Key sections to include:
- **Welcome Header:** A top section greeting the user, showing the organization name and some summary stats. In the current code, `ClientHeader` is used to display the org name, user, and sample stats like count of studies, active submissions, last login ¹⁶. Ensure this header is fed with real data if available (for now it might be dummy stats). It gives a quick context to the user about their overall status.
- **CRO Client Switcher (if applicable):** If the current user is a CRO admin managing multiple clients, show a section listing client organizations and a search bar to filter them ¹⁷. This was partly implemented: a grid of client cards that can be clicked to switch context ¹⁸. The `switchClient` function (from integration context) likely handles re-fetching data for that client ¹⁹. Ensure this works: when a CRO selects a client, update PortalContext and load that client's data (projects, etc.). If the user is not a CRO (i.e., a direct biotech client), this entire section should be hidden (already handled by an `isCRO` check in code ²⁰).

- **Project Portfolio (Main Panel):** Use the **ProjectManagerGrid** component to show an overview of projects the user is involved in ²¹. It was built to list a few key projects with progress bars, status, and a “Go to Work” button ²² ²³. Place this in the layout as the primary content on the left (taking two-thirds of width on desktop) ²⁴. If it’s not already, modify ProjectManagerGrid to accept props for filtering (like `userId` or `orgId`) so it shows projects relevant to the current context (the example passes `userId` and `orgId` ²⁵). For now, it uses dummy data ²⁶; integrate real data when backend is ready, but structure the component to easily switch to an API call later.
- **Quick Access Cards:** On the dashboard, provide at-a-glance panels for important functions:
 - *Vault Quick Access:* A small card listing recent documents and an upload shortcut. This was implemented as `VaultQuickAccess.jsx` ²⁷. It shows the last 2–3 documents with their names and types, and has a button to go to the full Vault module and another to upload a new document ²⁸ ²⁹. Ensure this component is imported and placed on the dashboard. In the layout, you can place VaultQuickAccess next to another card in a two-column secondary grid ³⁰.
 - *Analytics Quick View:* A card to show key metrics (e.g., submission velocity, delays, risk alerts). If not already created, implement `AnalyticsQuickView.jsx` similarly to VaultQuickAccess (placeholder content like charts or summary numbers). In the AI-guided plan, this was mentioned as the next component to build, focusing on showing things like number of days to deadline, any issues flagged, etc. Place it beside the Vault card in the dashboard grid ³⁰. These two cards give the user quick insights and actions (document management and overall analytics).
 - *Next Actions Sidebar:* A vertical list of tasks or notifications for the user, e.g., “Review Protocol draft” or “Submit annual report by June 1”. The `NextActionsSidebar` component is intended for this ³¹. It should occupy a right-side column on wide screens (one-third width) and list upcoming deadlines or to-dos. If not fully implemented, create a simple version that shows a static list of “next actions” for now (which can later be tied to tasks data). Each item might show a short description and possibly a link or button to jump to the related module.
- **Tabs for detail views:** To keep the dashboard clean yet powerful, implement a tabbed interface (as seen in code using a Tabs component ³²). Suggested tabs:
 - **Overview** – the main dashboard view as described above (this is the default tab, containing projects, quick cards, sidebar).
 - **Projects** – a tab that could list all projects in a detailed table or list (Step 6 will flesh this out). For now, it can simply show a placeholder text or reuse ProjectManagerGrid but with more entries.
 - **Documents** – a tab to show all important documents (maybe a table of recent documents or a link to the Vault module; could be a placeholder for now ³³).
 - **Activity** – a tab showing a log of recent activities across projects (this could reuse or link to a collaboration feed; again we see a placeholder for this in code ³³).
 - **Tutorial** – possibly include a Tutorial Guide tab (the code references a `TutorialGuide` component ³⁴) for onboarding new users. This fits the idea of an embedded guide to help users (especially given the complexity of regulated workflows).

Use a Tabs component or similar UI library to implement these, as the code snippet demonstrates. Ensure switching tabs doesn't lose context – e.g., keep the PortalContext the same; we're just toggling views.

- **Quick Links section:** The code example shows a “Quick Links” grid under the tabs ³⁵ – buttons linking directly to modules (Vault, Analytics, All Tasks, Settings). Keep this feature as it aligns with giving users direct navigation options. The links should correspond to the main modules or pages (and only show those allowed by context). For example, if a user doesn't have access to Analytics, maybe hide that button. Each button uses an icon and text, styled as an outline button with the brand color ³⁶ ³⁷.
- **Ensure data flows into the dashboard:** Wire up the components to use real context:
 - ProjectManagerGrid should use `orgId` from PortalContext (passed via props or context hook) to know which projects to show.
 - VaultQuickAccess should use the current org or study context to fetch documents (in its example, it fetched dummy docs via `securityService.getCurrentOrganization()` and then dummy data ³⁸ – later replace dummy with `fetch('/api/documents?orgId=...')` when backend ready).
 - NextActionsSidebar should ideally filter tasks by current user or study.
 - The AgentController (AI assistant trigger) is already included in the Overview tab ³⁹. Ensure it's working – it likely is a button or panel to open the AIAssistantPanel. The assistant should be context-aware from our integration: e.g., if the user is in an IND project, opening the assistant should add a message like “I see you're using the IND Wizard...” ⁴⁰ automatically.
- **Polish UI elements:** Make sure the dashboard components use the brand theme (they mostly have neutral backgrounds with shadows as seen in code ⁴¹ for card containers). Use consistent typography for headings (e.g., `<h2>` for section titles like “Vault Quick Access” ⁴², and maybe `<h3>` for sub-block titles). Icons from lucide-react are used widely for visual cues (check marks, clocks, etc. for statuses); ensure those icon components are included where appropriate (for example, project status could use a check icon if complete, etc. – in code, `ProjectManagerGrid` used colored bars rather than icons for status ²³, which is fine).

This dashboard is the **face of the portal**, giving at-a-glance info and one-click (or one-command) access to deeper functionality. Implementing it consolidates steps like “dashboard presets” (the quick cards) and sets the stage for more detailed project/task views. After this step, a user logging in should land on a rich overview page that is fully functional with placeholder or real data, ready to navigate anywhere else via clicks or the AI assistant.

Step 6: Develop Projects Hub and Detailed Views

Goal: Extend the portal to manage and display project information in detail. This involves creating pages for **All Projects** and possibly individual project workspaces, fulfilling the “Projects Hub” aspect of the enhancement plan.

- **All Projects Page (Projects Hub):** Implement a view that shows all projects accessible to the user in a comprehensive list or table. This could be a dedicated page (`ProjectsPage.jsx`) or reuse the Projects tab in the dashboard). Key features:
 - List each project with important attributes: name, status, type, client (if CRO user), percent complete, next due milestone, etc. Since `ProjectManagerGrid` already formats project cards, you can repurpose it in a different layout. For example, in the Projects tab content, you might simply call `<ProjectManagerGrid showAll={true} />` that displays all projects (not just a subset or not just in grid format).
 - Alternatively, create a more detailed table view for projects. For instance, a table with columns: **Project Name**, **Client** (for CROs), **Status**, **Completion %**, **Next Due**, **Actions**. This table can be scrollable for large project lists. Given the data is likely from an API (or dummy list for now), it might be easier to start with a simple map of projects to JSX as was done for the grid ²² ²³, and style it as a grid or list.
 - Provide filtering/sorting controls. For example, a dropdown or tabs to filter by status (Active, Completed, At Risk) or by study type (IND vs CSR projects). Initially, you can implement just a simple filter: e.g., a toggle to show only active projects. The interface might include a search bar to find a project by name (especially useful if there are many projects).
 - Pagination: If the number of projects is large, implement pagination or infinite scroll. In a demo or early stage, this may not be necessary, but structure the code so that adding a paginated API call later is straightforward.
- **Project Detail Workspace:** Decide how a user drills down into a project. Two likely approaches:
 - **Module-specific Routing:** If each project is strongly tied to a particular module (e.g., an IND project belongs in the IND Wizard module), then clicking “Go to Work” or the project name could navigate the user to that module’s interface, filtered to that project. For example, an IND project might take them to `/ind` route and load that IND’s data; a CSR project might go to `/csr` module. This requires that the modules themselves know what project to load, likely via a context or parameter (e.g., `studyId` or `projectId`). If using this approach, ensure the link encodes the project or study identifier (the example links to `/project/${projectId}` ⁴³, which suggests a generic route).
 - **Generic Project Page:** Alternatively, implement a generic project page (`ProjectWorkspace.jsx`) at route `/projects/:projectId` that displays an overview of the project and links to relevant module functions. For example, on a project page for “IND-2025-034”, show its summary (description, status, completion, missing items) and have buttons like “Open IND Wizard” (if relevant)

or "View Documents". This page could also show project-specific tasks and team members. This approach is useful to have a central place even if multiple modules contribute to one project.

You can choose a hybrid: initially route to a generic page, and from there allow navigation into modules. Since time is a factor, a simple implementation is: clicking a project in the list navigates to `/projects/:id` and that page just identifies which module or next step to go to. For instance, if `id` starts with "IND", you could auto-redirect to the IND Wizard route.

- **Integrate with PortalContext:** Ensure that when navigating into a specific project or study, the relevant context is updated. For example, if `project = study`, set `portalCtx.studyId = ...` and perhaps `programId` / `orgId` if not already. This way, components or the AI assistant know which context is active. The `ModuleIntegration` could also share `activeProject` context.
- **UI enhancements:** On project cards in the grid or list, use visual cues to highlight status:
 - If a project is at risk or has missing items, emphasize it (e.g. red text or an alert icon). The dummy data uses `status === 'at_risk'` to color the progress bar red ⁴⁴. Continue such logic and possibly add an icon or label "At Risk".
 - Completed projects can be de-emphasized (greyed out or grouped separately).
 - The warm luxe palette can be applied: e.g., instead of bright red/green/yellow for status, use softer brand-consistent variants (maybe gold for in-progress, emerald for complete, rust for at-risk – depending on brand guidelines). Tailwind configuration can map `bg-yellow-500` etc., to brand-specific hex values to maintain consistency.
- **Permissions:** A CRO user might see projects from multiple organizations, whereas a biotech client only sees their own. Ensure that the Projects API (if any) is called with the right context (for CRO maybe an endpoint that fetches all orgs' projects or one org at a time). In the current dummy implementation, they took `org.type` to decide layout (5 columns for CRO vs 3 for biotech) ⁴⁵. In our context, ensure that if `userRole` is CRO Admin, you label projects by client (the dummy data includes a `client` field for each project ⁴⁶). Display the client name in the project list for clarity (as done in the card: `project.client` is shown ⁴⁷).
- **Add Project capability:** For an Admin or PM role, allow creation of a new project. On the Projects page, include an **"Add Project"** button (maybe only visible to certain roles). This could open a modal form asking for project name, type, associated program/study, etc. Since backend integration might be complex, this can be a placeholder that simply adds to the local list for now. However, including it strengthens the impression of a full project management hub. (Note: The portal landing had an "Add Client" for CROs ⁴⁸; similarly, an "Add Project" for a specific org's PM could be in the Projects tab or on the Program/Study view.)

By implementing the Projects Hub, users can navigate and oversee all projects in one place, fulfilling the need for a **convergent workspace** for project management. This step ties closely with the next (tasks), as projects and tasks are two sides of managing the workflow. Ensure that the design here leaves hooks or

space to display tasks related to each project, either on the project list (like showing how many tasks open per project) or on a project detail page.

Step 7: Implement Collaboration Shell Features

Goal: Add collaboration-oriented features to the portal – enabling both AI-human collaboration and human-human teamwork. This “collaboration shell” wraps around the modules to provide chat, guidance, and shared activity context.

- **AI Assistant Integration:** The **AI assistant panel (Claude Opus 4.5)** is a key collaborative element. Make sure it is fully incorporated:
- The `AgentController` component on the dashboard presumably controls toggling the assistant ³⁹. It likely opens the `AIAssistantPanel` (which contains the chat UI). Ensure that `AIAssistantPanel` is mounted, perhaps at the root of `PortalShell` or in a context where it can overlay on any page. The assistant should be accessible from anywhere in the portal via a consistent UI element (like a chat icon fixed at bottom-right).
- **Context-awareness:** Verify that the assistant receives context from the integration layer. E.g., if a user is currently viewing a particular project or module, the assistant’s context injection (those system messages that say “I see you’re in CSR Intelligence...” ⁴⁹) should trigger correctly. From Step 3, we shared context on organization and potentially active module; extend that if needed to share “activeProject” or “lastAction” so the assistant can help accordingly.
- **Cache-first AI:** The assistant’s logic (in `RegulatoryIntelligenceCore.getChatResponse`) might already handle some caching or at least we can simulate it. Reinforce this by maybe caching the last N Q&A pairs at the integration layer so if a user repeats a question, the assistant could retrieve the previous answer quickly. This is a bonus – mainly ensure performance by not re-fetching unchanged context each time (the `getRelevantContext` function in the code already sifts out the latest context item to send ¹¹).
- **Collaboration/Chat for Users:** Introduce a feature for users to collaborate. Depending on project scope, this could be:
 - **Comments/Notes:** Allow users to comment on projects or documents. For example, on a project detail page, include a comments thread where team members can discuss. This requires a backend or at least a placeholder data store. You might create a simple context or array to hold comments per project for the demo.
 - **Mentions/Notifications:** Allow tagging a user (e.g., “@Dr.Smith please review section 2”). While a full implementation is complex, the UI could have an input and display of messages, and you’d integrate with email or notifications later.
 - **Activity Feed:** Use the **Activity log** tab (from Step 5) to show a chronological feed of events (e.g., “Protocol document updated by Alice (2h ago)”). The code already anticipates an Activity view ³³. You can populate this feed from dummy data or from actual actions (if you log when a user uploads

a document or completes a task). At minimum, simulate a few events so the tab isn't empty – this gives a sense of collaboration history.

- **Collaboration UI Shell:** Consider a dedicated component (e.g., `CollaborationSidebar` or `CollaborationPanel`) that can slide out. This could unify AI chat and user chat in one interface, or separate but accessible via icons. For instance, a tab in that panel for AI Assistant, another tab for Team Chat. If building such a UI is too much at this stage, at least ensure the placeholders exist (so an engineer could expand it later).
- Perhaps repurpose the `NextActionsSidebar` area or the right column: that could double as a collaboration panel when needed. Alternatively, a modal approach: clicking a “Chat” icon could open a modal with either the AI chat or a team chat.
- **Role-based collaboration permissions:** Respect roles in collaboration features. For example, maybe only Admins and Editors can post comments, whereas Viewers can only read them. Or restrict certain channels to internal use (a CRO might have an internal note on a client’s project not visible to the client user). These constraints tie back to our `PortalPolicy` roles. In UI, implement simple checks like:

```
{portalCtx.userRole !== 'Viewer' && <CommentInput ... />}
```

to hide input for read-only users. It’s straightforward to adjust once roles are in context.

- **Warm luxe experience in collaboration:** Ensure even chat windows or comment threads follow the design principles. For instance, the AI assistant panel uses a modern chat UI with motion animations (framer-motion) and icons for user/assistant ⁵⁰ ⁵¹. Continue using such libraries to make the collaboration feel smooth and polished. Use brand colors for chat bubbles or highlights (perhaps assistant messages in a tinted background, user messages in another, aligning with brand palette).

By introducing these collaboration features, the portal becomes not just a static workspace but an **interactive, multi-user environment**. This fosters real-time teamwork and guidance, which is crucial in complex regulatory projects. After this step, users (and the AI) can communicate contextually within the portal – whether it’s asking the AI a question or leaving a note for a colleague – all without leaving the platform.

Step 8: Integrate Task and Workflow Management

Goal: Build out the **task/project work management** features, allowing users to track and manage tasks (Next Actions) across studies and projects. This step ensures the portal not only displays information but also supports the users in doing their work and checking it off.

- **NextActionsSidebar completion:** Finalize the Next Actions sidebar component so it dynamically displays the user’s tasks. If not already implemented, design a structure for tasks:

```
interface Task {
  id: string;
  title: string;
  projectId: string;
  dueDate?: string;
  status: 'open' | 'completed' | 'overdue';
  assignedTo?: string; // user id or name
  context?: string; // e.g. "Protocol IND-2025-034"
}
```

For demo purposes, you can populate a list of tasks in state or fetch from an endpoint (e.g., `/api/tasks?userId=...`). Display tasks in the sidebar as a list of clickable items: - Overdue or urgent tasks could be highlighted (e.g., red text or an "!" icon). - Each task item could show a short title and perhaps due date (e.g., "Finish CSR Draft – due Jun 5"). - Clicking a task should navigate to the relevant context: e.g., if a task is "Finish CSR Draft", it might open the CSR Intelligence module for that project. This can leverage the intent engine or direct linking (you might encode in the task's context or ID which module to open).

- **All Tasks View:** Implement the **"All Tasks"** page or tab (accessible via the Quick Link or a main nav if appropriate) ⁵². This view would list all tasks the user can see. If the user is an admin or project manager, maybe they see all tasks for their team; if just a regular user, only their tasks. This page can be similar to the Projects list but for tasks:
 - Columns: Task, Project, Due Date, Status, Assignee.
 - Allow marking tasks complete (perhaps a checkbox or a "Mark Done" button for each open task).
 - Possibly allow editing tasks (change due date, reassign) for authorized roles.
 - Filtering: e.g., filter by project or by status (show only open tasks, or only tasks due this week, etc.).

Since implementing a full backend is outside scope, maintain tasks in React state or context for now. Actions like marking complete can just update state (and maybe remove the task from open list or move to a "completed" list).

- **Task creation and linking:** Provide ways to create tasks:
 - On the tasks page, an "Add Task" button that opens a form (fields: title, project (choose from projects list), due date, assignee).
 - On a project page, maybe a smaller "New Task" input to quickly add a task related to that project.
 - Integrate with other modules: e.g., on a document view, a button "Create Task from this document" could prefill a task like "Review [DocumentName]".

For now, stub these forms out and log to console or update local state. The existence of the UI and the flow is more important than full persistence, given this is a front-end plan.

- **Visualize progress:** Consider adding a **Kanban or timeline** element for tasks if time permits. For instance, show tasks grouped by status or a calendar view for due dates. This can remain a

placeholder graphic or simple text for now. The main idea is to convey that the portal offers some **workflow visualization**, not just static lists (which is valuable for an enterprise user experience).

- **Integration with AI for tasks:** Leverage the AI assistant in task management:
 - The assistant can generate tasks or checklists. For example, if a user asks, “What are the next steps for IND-2025-034?”, the assistant might respond with a list of tasks. If it does, we could allow the user to click a button to **“Add these tasks to my project”**. While implementing that fully is complex, design with it in mind – maybe the assistant’s response could have structured data.
 - The user could also ask the AI to mark a task as done or create a new task. If we have the intent engine set up, phrases like “mark task 123 complete” could be parsed and trigger a state update in the app. Document such patterns in the Intent Engine mapping (Step 4).
- **Ensuring regulatory compliance in tasks:** In regulated domains, tasks might need audit trails (who completed what when). We won’t implement audit logging here, but mention that completing a task could send a confirmation to backend to timestamp it. UI-wise, maybe show the completed date on done tasks.
- **User roles and tasks:** Respect role differences – e.g., maybe only Admin/Editors can create or assign tasks, whereas Viewers only see them. The UI should hide assignment controls from those without permission. For CRO vs client, possibly a CRO PM could assign tasks to client users and vice versa. That complexity can be handled later, but our policy definitions can anticipate it (like tasks module accessible to Editors and Admins, read-only to Viewers).

After integrating tasks, the portal supports active **work management**. Users can not only view project statuses but also act on them (through tasks, checklists, and the AI suggestions). This closes the loop between *information* and *action*, which is critical for productivity. With this step, the “Next Actions” part of the UI Enhancement is realized in both the dashboard (sidebar) and dedicated views.

Step 9: Finalize Admin Console and Controls

Goal: Complete the **Admin Console** features, allowing management of organizations, users, roles, and feature toggles. This step builds on the admin scaffolding from earlier (like user invitations) and ensures the portal is configurable for enterprise needs.

- **Organization Management:** Expand the admin interface to handle organizations (clients):
 - The portal landing already provides an **“Add Client”** button for CRO admins ⁴⁸. Implement the onClick to open a form (or navigate to an `/admin/new-client` page). The form can gather organization name, region, industry, etc. On submit, call a backend API (`POST /api/admin/orgs`) to create the org, or if no backend, simply log and add to a local list for display. After creation, the new org should appear in the client list (perhaps the admin can then switch into it to set up users).
 - Provide an `/admin/orgs` page listing all organizations. This could be a simple list with options to view details or remove an org. Removing/deactivating could just be simulated by a confirmation dialog (and perhaps a note that it’s not implemented fully without backend).

- If applicable, allow editing an organization's details (name, type). For example, if a biotech client changes name or if we need to set an org's regulatory region (which could be used for PortalPolicy filtering), allow that in an edit form.

- **User & Role Provisioning:** Use the structures from Sprint-7 to list and invite users:

- The route `/admin/org-users` (as added in Sprint-7) should be integrated into PortalShell's route structure (e.g., accessible under Admin menu) ⁵³. It shows a table of users in the current org with their roles ⁵⁴. Ensure that when an admin navigates here, `PortalContext.orgId` is set (they should select an org first). If not set, perhaps redirect to pick an org.
- The component `AdminOrgUsers` fetches users for the org and displays them ⁵⁵. Make sure it uses the new AuthContext for token and PortalContext for orgId (as in the snippet) ⁵⁶. This is likely already functional from the code.
- Implement the **Invite User** form in that page: fields for email and role, and a button to invite ⁵⁷. On submit, call the POST invite API ⁵⁸. In our environment, ensure the API URL is configured (`VITE_API_URL` env variable). If the backend is running (Supabase or Node as indicated in code), this will actually create a user entry. If not, simulate by immediately adding to the local `users` state for demo.
- Add ability to change roles: perhaps the user list could have a dropdown per user to select a new role (Admin/Editor/Viewer). On change, call an API (`PUT /api/admin/orgs/:orgId/users/:userId` to change role) or simulate it by updating state. This ensures that roles can be managed post-invite.
- Attach these operations to real backend endpoints if available. The code gave SQL for roles and a basic invite route ⁵⁹ ⁶⁰, so presumably `/api/admin/orgs/:id/invite` and the GET for users exist. Use them if possible; otherwise, dummy data.

- **Feature Toggles / Module Management:** Provide admin control over modules:

- Add an Admin page (maybe `/admin/modules` or part of org settings) where an admin can enable/disable certain modules for a given organization. For example, a toggle for "Enable Analytics Module" or "Enable AI Assistant" per client. This would allow the company to roll out features gradually or tailor the portal to client needs.
- Implement this by showing a list of modules (from our ModuleConfig list) with checkboxes or switches. Persisting this setting could be to a config object in backend (not trivial to do fully here). Instead, demonstrate by storing an override in PortalPolicy (e.g., a context or a simple in-memory object that admin changes). When rendering modules, consult this override in addition to the static conditions.
- This feature might be more forward-looking, but mentioning and partially implementing it shows a thorough admin console. It also aligns with a **polymorphic** concept: different clients might have different module sets (e.g., a Med Device client might not use IND Wizard at all – an admin could turn it off for them).

- **Audit Trail & Logging:** While building a full audit log UI may be outside scope, at least prepare for it:

- Perhaps include an “Activity Logs” section under Admin where all system events are listed (could reuse the collaboration activity feed but with system-level events).
- Show events like user invites, role changes, logins, etc. For now, this can be a static list or pulled from wherever activity data is stored. The presence of this section indicates the platform’s readiness for compliance audits (important in regulated environments).
- **Admin UI integration:** Add an **Admin** entry in the navigation (sidebar or top menu) that is only visible to users with Admin roles. For example, if `portalCtx.userRole === 'Admin' && isCRO` then show “Admin Console” in the nav. Under this section, you might have sub-links: “Organization Users”, “Organizations”, “Settings/Modules”. We saw in Sprint-7 they added a route `/admin/org-users` to PortalShell routes ⁵³. We should continue that pattern for any new admin pages.
- **Security check:** The UI should defensively check roles for admin pages. It’s possible a non-admin could manually navigate to an `/admin` URL; in such case, use a redirect or message. For example, in AdminOrgUsers component, if the user’s role isn’t Admin, either redirect them out or show “Unauthorized”. This is on top of backend protection (which presumably exists via middleware ⁶¹).

By rounding out the admin console, the portal can be properly administered and configured, which is crucial for enterprise deployment. This step ensures the **UI Shell** isn’t just user-facing but also allows **back-office control**. The combination of user management, org management, and module toggling gives Concept2Cure flexibility to support different clients and maintain compliance (assigning correct roles, etc.). At this point, the UI Enhancement project’s admin aspects (Org & Role Provisioning, Admin console) are implemented.

Step 10: UI Hardening, Testing, and Performance

Goal: Polish the portal UI for production readiness. This involves thorough testing, fixing any UI inconsistencies, optimizing performance (especially around AI integration), and ensuring the design is robust and accessible. This final step is crucial for delivering a **regulatory-grade** product with modern quality standards.

- **Cross-Browser/Cross-Platform Testing:** Verify the portal on all major browsers (Chrome, Firefox, Safari, Edge) and ensure consistent behavior. Fix any CSS issues that appear (e.g., flex or grid differences). Also test various screen resolutions:
- On smaller laptops, does the dashboard grid reflow nicely? (Use responsive design techniques like the existing `md:grid-cols-2` etc. classes ³⁰).
- On tablets or small screens, maybe implement a collapsible sidebar if not done (e.g., hamburger menu to toggle the side nav).
- Ensure important content is not cut off or overflowed. Scrollbars should appear for long lists (the SideNav had `overflowY: 'auto'` in code ³, keep such things).
- **Accessibility (A11Y):** Go through the app and add accessibility attributes:
- All interactive elements (buttons, links) should have discernible text or `aria-label`. For icon-only buttons (like a possible “Close chat” X icon), add `aria-label="Close chat"`.

- Ensure color contrast meets WCAG guidelines – the warm luxe palette should still have sufficient contrast (check that text on background passes AA at least; adjust colors if needed).
- Tab order: can a user navigate via keyboard? The sidebar links, header menu, and any form fields should be reachable by Tab key in a logical order. Use outlines or highlights on focus states to make keyboard navigation visible.
- For dynamic content (like when the assistant panel appears or modals open), manage focus appropriately (e.g., focus the first input in a dialog, and trap focus within it until closed).

• **Performance Optimizations:**

- **Bundle Splitting:** Ensure that rarely-used modules (like maybe the Admin console or heavy analytics charts) are code-split. Use React lazy import for such routes so they don't bloat initial bundle.
- **Caching & Network:** Audit network calls. Implement caching where feasible. For example, if the portal fetches organizations list on every load, consider caching it in localStorage or in a React context that persists across sessions (assuming org list doesn't change often). Similarly, cache static reference data (like list of regulatory agencies or product types).
- For the AI assistant, implement a simple cache for recent Q&A. If a question has been asked before (exact match), the assistant can return the stored answer immediately (while perhaps still verifying/ updating in background). This could be as simple as a dictionary in memory for this session. This demonstrates the **cache-first** approach in AI integration.
- Investigate any slow parts of the UI (for example, loading a large project list). Use lazy rendering or virtualized lists if needed to handle hundreds of items efficiently.

• **Stability and Error Handling:**

- Add error boundaries in React so that a failure in one module doesn't break the whole app. For instance, wrap module routes in an ErrorBoundary that shows a user-friendly message if something goes wrong.
- Handle API errors gracefully: e.g., if fetching projects fails, show an error message on the dashboard card (like "Failed to load projects") instead of leaving a blank area. The code already logs errors to console ⁶²; extend that to UI feedback.
- Ensure the AI assistant handles errors from the API elegantly – in the code, they catch errors and add an error message to the chat ⁶³, which is good. Keep such patterns.

• **UI Consistency Audit:**

- Go through each screen for consistent typography, spacing, and colors. For example, headings should have uniform styles (maybe define a Tailwind or CSS class for `h2` in panels). Align buttons styles (use the same button component or class for all primary actions).
- Check all icons and visuals: ensure they align with the text and each other. In ProjectManagerGrid, the progress bar and status text alignment should look polished; adjust margins as needed.
- Remove any temporary or debug elements (console.logs, placeholder text like "will be displayed here" ³³). Replace placeholders with either actual data or a clean notice ("No data available" when appropriate).

- Harden the “No data” states: for example, if an org has no projects, ProjectManagerGrid should display a friendly “No projects yet” message instead of showing an empty grid. We saw VaultQuickAccess handles no documents with a message ⁶⁴ – do similar for other components.
- **Compliance Checks:** Since this is for regulated domains, consider adding compliance-related UI elements:
 - Display version or environment info somewhere (so users know if they’re on a validated version of the software).
 - Ensure date/time formats meet standards (e.g., use ISO or a clear format for global use).
 - If any session timeout or disclaimer banners are needed (common in pharma apps), plan where to include them (e.g., a footer with “Confidential” notice, etc., if required).
- **Documentation & Handoff:** As a final part of hardening, update documentation:
 - The repository README should explain how to run the app (especially important since we assume a fresh Codespace scenario).
 - If using environment variables (for API URLs, etc.), list them and their purpose.
 - Document the module registry and how to add new modules following the established pattern (so future devs or AI agents can continue expanding the portal).
 - Optionally, prepare user-facing documentation for the portal’s features – even if just a draft, this helps ensure all features are discoverable and understandable (this might be outside the developer agent’s scope, but noting it is good practice).

After Step 10, the portal should be **robust, fast, and user-friendly**, ready for real-world use. It incorporates all the enhancements (1–10) in a cohesive manner and has been refined to meet high standards of quality. The next-generation Concept2Cure client portal will thus be stable and scalable for deployment across biotech, pharma, and device clients.

Execution in Codespaces

Each of the above steps can be executed as a sequence of development tasks in a GitHub Codespace, utilizing AI assistance (Copilot/Claude) for boilerplate but with careful human (or agent) review:

1. **Environment Setup:** Launch a Codespace with the Concept2Cure repository. Install dependencies and start the development server to verify baseline functionality.
2. **Follow the Protocol Steps:** Proceed through steps 1 to 10 in order. For each step:
3. **Implement code changes** as described (create or modify files, write interfaces, build components, adjust routes). Use the file paths given (e.g., create `portalPolicy.ts`, update `PortalShell.tsx`, etc.). You can copy code snippets from this protocol (like the PortalContext example ⁹ or others) and adapt them.
4. **Test the app** in the Codespace after each step. For example, after Step 2, run the app and ensure the PortalShell with nav renders and you can navigate to the dashboard route without errors. After Step 5, verify the dashboard UI elements appear and respond (maybe with dummy data).
5. **Use AI as needed:** GitHub Copilot can expedite writing repetitive code (like generating interface properties or basic JSX loops). Claude (this document’s consumer) can be prompted step-by-step to

generate components (similar to how VaultQuickAccess was created ⁶⁵ ²⁸). However, always verify the AI-generated code against the requirements here.

6. **Commit changes** with clear messages ("Step 3: Added PortalContext for multi-tenant state", etc.). This aligns with an iterative build and makes it easier for the engineering lead to review progress.

7. **Final Testing:** After implementing all steps, do a full run-through in the Codespace:

8. Create a sample scenario (e.g., a CRO admin with one client org having two projects). Click through the UI: switch orgs, open projects, try the assistant, add a task, etc. Make note of any issues and fix them.

9. If possible, run automated tests or linting to catch any errors.

10. The engineering lead might also run their own tests; be prepared to address any feedback (e.g., adjust spacing, fix a broken link).

11. **Deployment Prep:** Since the portal should run from scratch, ensure that any build scripts (`npm run build`) work in the Codespace. Resolve any build warnings (especially with TypeScript interface exports or unused variables).

12. Given that only the repo and agent are online, all external calls (like fetches to API) should have fallbacks or be mocked so the app doesn't crash in a demo if the backend isn't reachable. For instance, wrap `fetch` calls in try/catch and use dummy data if network fails, to allow the front-end to still showcase features.

Following this structured protocol, Developer Agent Claude 4.5 can systematically implement and refine the Concept2Cure convergent portal. The result will be a **modular, intelligent, and polished** client portal UI that aligns with both the innovative **Convergent Canvas** design philosophy and the practical needs of regulated industry projects.

Document Formats: This build protocol is formatted in **Markdown** for clarity and easy copy-paste into development notes. A `.docx` (Word document) variant can be generated for sharing with stakeholders or for formal documentation – the sections and content remain the same. (To obtain the Word version, you can convert this Markdown using a tool or copy it into a Word processor; all headings and bullet points will be preserved in the export.)

By delivering the protocol in both developer-friendly (Markdown) and business-friendly (Word) formats, we ensure that the plan is accessible to the AI executor and the human engineering lead alike, enabling seamless implementation and review. ⁹ ²⁵ ⁵⁵ ⁶⁴

¹ ¹¹ ⁴⁰ ⁴⁹ ⁵⁰ ⁵¹ ⁶³ AIAssistantPanel.jsx

<https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/client/src/components/AIAssistantPanel.jsx>

2 3 4 7 8 9 13 14 Pasted--TrialSage-Client-Portal-Sprint-6--1745712916988.txt

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/
Pasted--TrialSage-Client-Portal-Sprint-6--1745712916988.txt](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/Pasted--TrialSage-Client-Portal-Sprint-6--1745712916988.txt)

5 6 10 ClientPortal.jsx

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/client/src/components/
client-portal/ClientPortal.jsx](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/client/src/components/client-portal/ClientPortal.jsx)

12 53 54 55 56 57 58 59 60 61 Pasted--TrialSage-Admin-Panel-Sprint-7-Or-1745713051870.txt

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/
Pasted--TrialSage-Admin-Panel-Sprint-7-Or-1745713051870.txt](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/Pasted--TrialSage-Admin-Panel-Sprint-7-Or-1745713051870.txt)

15 16 17 18 19 20 21 24 25 30 31 32 33 34 35 36 37 39 48 52 62 ClientPortalLanding.jsx

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/client/src/components/
client-portal/ClientPortalLanding.jsx](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/client/src/components/client-portal/ClientPortalLanding.jsx)

22 23 26 43 44 45 46 47 Pasted--client-src-components-ProjectManagerGrid-jsx-import-React-useEffect-useState-from-reac-1745806472720.txt

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/
Pasted--client-src-components-ProjectManagerGrid-jsx-import-React-useEffect-useState-from-reac-1745806472720.txt](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/Pasted--client-src-components-ProjectManagerGrid-jsx-import-React-useEffect-useState-from-reac-1745806472720.txt)

27 28 29 38 41 42 64 65 Pasted--Perfect-You-said-Next-so-we-move-forward-carefully-Now-I-ll-build-the-next-component-1745806610255.txt

[https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/
Pasted--Perfect-You-said-Next-so-we-move-forward-carefully-Now-I-ll-build-the-next-component-1745806610255.txt](https://github.com/concept2cure/ClinicalSageAI-2/blob/4e65d9ad72e52c7803bec3cf47bd7b451e94408d/attached_assets/Pasted--Perfect-You-said-Next-so-we-move-forward-carefully-Now-I-ll-build-the-next-component-1745806610255.txt)